

MATLAB基础知识

一、数据类型

1. 数值型

1.1 整数类型

- 除int64和uint64类型之外，所有整数类型都可以进行数学运算。
- 由于MATLAB中数值的默认存储类型是双精度浮点数类型，因此，在变量设置为整数时，需要使用相应的转换函数。转换函数为 `相应的数据类型 ( )` （如： `a = int16(120)` ）

函 数	运 算 法 则	示 例
floor(x)	向下取整	floor(1.2)=1
ceil(x)	向上取整	ceil(1.2)=2
round(x)	取最接近的整数；如果小数部分是0.5，则向绝对值大的方向取整	round(-2.5)=-3
fix(x)	向0取整	fix(-1.2)=-1

1.2 浮点数类型

- 双精度浮点数在参与运算时，返回值的类型依赖于参与运算的其他的数据的类型。
 - 当参与运算的其他数据为逻辑型、字符型时，返回值为双精度浮点数类型。
 - 当参与运算的其他数据为整数类型时，返回值为相应的整数类型。
 - 当参与运算的其他数据为单精度浮点数类型时，返回值为相应的单精度浮点数类型。

💡 在MATLAB中，整数类型不能和单精度浮点数类型进行算术运算。

1.3 复数

复数包括实部和虚部两部分。MATLAB中默认使用字符i或j作为虚部标志。在创建复数时，可以直接按照复数形式进行输入或使用complex函数。

函 数	说 明	函 数	说 明
real(z)	返回复数的实部	imag(z)	返回复数的虚部
abs(z)	返回复数的模	angle(z)	返回复数的辐角
conj(z)	返回复数的共轭复数	complex(a,b)	创建复数

1.4 无穷量和非数值量

在MATLAB中使用inf和-inf代表正无穷量和负无穷量；用NaN代表非数值量。

正负无穷量一般是由运算溢出超出双精度浮点数类型能表示的数值范围产生的；而非数值量则是由0/0或inf/inf产生的，这两种NaN彼此是不相等的。

2. 逻辑型

逻辑型数据是指布尔类型数据，包括真（True）、假(False)两种数值，用于表达数据之间的逻辑关系。

作为所有关系式和逻辑表达式的输入，MATLAB把任何非零数值都当做真，把零当作假。所有关系表达式和逻辑表达式的输出：对于真，则为1；对于假，则为0。

- MATLAB中的关系运算符：

关系运算符	说明	关系运算符	说明
<	小于	>=	大于或等于
<=	小于或等于	==	等于
>	大于	~=	不等于

MATLAB中的关系运算符能用来比较两个同样大小的数组，或者一个数组和一个标量。在后一种情况当中，标量和数组中每个元素相比较，结果与数组大小一样。

- MATLAB中的逻辑运算符：

逻辑运算符	说明	逻辑运算符	说明	逻辑运算符	说明
&	与		或	~	非

除了上述关系运算符与逻辑运算符，MATLAB还提供了大量其他关系函数与逻辑函数：

- xor(x,y)指令的功能为异或运算，x和y同为0或非零的时候返回0，否则返回1。
- any(x)指令的功能为判断x是否为零向量或零矩阵，如果是非零向量或非零矩阵，则返回1，否则返回0。

除此之外，MATLAB还提供了大量的函数，在运算过程中用来测试特殊值或条件是否存在，并返回相应的表示结果的逻辑值。

函数名称	函数功能	函数名称	函数功能
isfinite	元素有限，返回真值	ishold	当前绘图保持状态是‘ON’，返回真值
isempty	参量为空，返回真值	isspace	元素为空格字符，返回真值
isnan	元素为不定值，返回真值	isstring	参量为一个字符串，返回真值
isinf	元素无穷大，返回真值	isstudent	MATLAB为学生版，返回真值
isreal	参量无虚部，返回真值	isletter	元素为字母，返回真值

3. 字符型

字符串一般是ASCII码的数值数组，作为字符串表达式进行显示。

4. 函数句柄

在MATLAB中，对函数的调用方法分为直接调用法和间接调用法。

- 对于直接调用法，被调用的函数通常被称为子函数。但是子函数只能被与其M文件同名的主函数或M文件中的其他函数调用，一个文件中只能有一个主函数。
- 使用函数句柄对函数进行调用可以避免上述的问题。创建函数句柄需要用到操作符@。MATALB函数库提供的各种M文件中的函数和使用者自主编写的程序的内部函数命都可以被创建函数句柄，从而可以通过函数句柄来实现对这些函数的间接调用。

创建函数句柄的一般语法形式如下：

```
Function_Handle = @Function_filename;
```

```
F_Handle = @cos
```

```
F_Handle = 包含以下值的 function_handle:  
    @cos
```

```
x = 0 : 0.25 * pi : 2 * pi;  
F_Handle( x )
```

```
ans = 1×9  
    1.0000    0.7071    0.0000   -0.7071   -1.0000   -0.7071   -0.0000    0.7071    1.0000
```

MATLAB中提供了大量关于函数句柄的操作函数，将函数句柄与其他数据类型联系起来，扩展了函数句柄的应用。

函数名称	函数功能
s = functions(fh)	返回一个结构体s，存储函数的名称、类型（simple或overloaded），以及函数M文件的位置
c = func2str(fh)	将函数句柄fh转换为函数名称的字符串
fh = str2func(str)	将字符串str代表的函数转换为函数句柄
save(fname.mat,fh)	将函数句柄fh保存在*.mat文件中
load(fname.mat,fh)	把*.mat文件中存储的函数句柄fh加载到工作区中
isa(var,"function_handle")	检测变量var是否是函数句柄
isequal(fhA, fhB)	检测两个函数句柄是否对应同一个函数

函数句柄的简单操作函数

```
F_Handle = @cos
```

```
F_Handle = 包含以下值的 function_handle:  
    @cos
```

```
F_Handlea = @exp
```

```
F_Handlea = 包含以下值的 function_handle:  
    @exp
```

```
str = "F_Handleb"
```

```
str = "F_Handleb"
```

```
functions(F_Handle)
```

```
ans = 包含以下字段的 struct:  
    function: 'cos'  
        type: 'simple'  
        file: ''
```

```
func2str(F_Handle)
```

```
ans = 'cos'
```

```
str2func(str)
```

```
ans = 包含以下值的 function_handle:  
    @F_Handleb
```

```
isa(F_Handlea,"function_handle")
```

```
ans = logical  
     1
```

```
isa(str,"function_handle")
```

```
ans = logical  
     0
```

```
isequal(F_Handlea,F_Handle)
```

```
ans = logical  
     0
```

5. 结构体

一个结构体可以通过字段存储多个不同类型的数据。因此，结构体相当于一个数据容器，把多个相关联的不同类型的数据封装在一个结构体对象中。

创建结构体的方法有两种，第一种是通过字段赋值创建结构体

结构体—通过字段赋值创建结构体

```
Student.Name = "Petter";  
Student.Age = 18;  
Student.Grade = 9;  
Student.Subject = {'Chinese','Math','English'};  
Student.Result = {95,100,99};  
Student
```

```
Student = 包含以下字段的 struct:  
    Name: "Petter"  
    Age: 18  
    Grade: 9  
    Subject: {'Chinese' 'Math' 'English'}  
    Result: {[95] [100] [99]}
```

```
whos|
```

Name	Size	Bytes	Class	Attributes
Student	1x1	1643	struct	

第二种是利用struct函数创建结构体，语法形式如下：

```
StrArray = struct('field1', value1, 'field2', value2, ...)
```

```
teacher = struct('name', 'Mary Smith', 'subject', 'Mathematics', 'years', 10);
teacher
```

teacher = 包含以下字段的 struct:
name: 'Mary Smith'
subject: 'Mathematics'
years: 10

```
students = struct('name', {'John', 'Jane'}, 'age', {20, 21}, 'grades', {{85, 92}, {88, 90}});
students
```

students = 1x2 struct

字段	name	age	grades
1	'John'	20	1x2 cell
2	'Jane'	21	1x2 cell

```
students(1)
```

ans = 包含以下字段的 struct:
name: 'John'
age: 20
grades: {[85] [92]}

```
students(2)
```

ans = 包含以下字段的 struct:
name: 'Jane'
age: 21
grades: {[88] [90]}

```
teachers(2) = struct('name', 'Mary Smith', 'subject', 'Mathematics', 'years', 10);
teachers
```

teachers = 1x2 struct

字段	name	subject	years
1	[]	[]	[]
2	'Mary Smith'	'Mathematics'	10

```
teachers(1)
```

ans = 包含以下字段的 struct:
name: []

subject: []

years: []

6. 数组类型

在MATLAB中进行运算的所有类型数据都是按照数据及矩阵的形式进行存储和运算的，二者在MATLAB中的基本运算性质不同：数组强调元素对元素的运算，而矩阵则采用线性代数的运算方式。

数组的属性以及数组之间的逻辑关系是编写程序是非常重要的两个方面。数组的元素可以是任何类型。利用数组的构建方法可以直接对变量进行赋值。

数组可以直接通过变量进行赋值来创建，用中括号。

```
a = [1 2 3 4 5 6];
a
```

a = 1×6

1	2	3	4	5	6
---	---	---	---	---	---

Array = i : j : k，这种形式中，可以使用“:”来代表一系列数值，表示从i开始，步长为j，到k结束的数字序列。

```
b = 2:2:100;
b
```

b = 1×50

2	4	6	8	10	12	14	16	18	20	22	24	26	28	30	32	34	36	38	...
---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----

还有一些预定义的函数也可以用来创建线性序列和逻辑序列linspace(a,b,n)表示在区间[a,b]上创建一个含有n个元素的向量。|

```
c = linspace(1,100,100);
c
```

c = 1×100

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	...
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	-----

7. 元胞数组

7.1 概述

组成元胞数组的每一个元素称作一个单元。每个单元可以包括一个任意数组（如数值数组、字符串数组、结构体数组或另外一个元胞数组）。MATLAB中使用元胞数组的目的在于，它可以把不同类型的数据归并到一个数组中。

元胞数组的创建方式有两种。

元胞数组

1. 使用赋值语句创建元胞数组

与一般数组不同的是，元胞数组使用花括号{}来创建，使用逗号“,”或空格来分隔每个单元，使用“;”来分行。

```
%创建元胞数组
c = {"x",[1;3;6];10,pi};
c
```

c = 2×2 cell

	1	2
1	"x"	[1;3;6]
2	10	3.1416

2. 利用cell函数创建空元胞数组

与一般的数值数组一样，元胞数组的内存空间也是动态分配的。因此，使用cell函数创建空元胞数组的主要目的是为该元胞数组**预先分配连续的存储空间**。

`a = cell(2,2)` %创建一个2*2的空元胞数组

`a = 2x2 cell`

	1	2
1	[]	[]
2	[]	[]

`b = cell(3)`

`b = 3x3 cell`

	1	2	3
1	[]	[]	[]
2	[]	[]	[]
3	[]	[]	[]

7.2 元胞数组的寻访

在元胞数组中，单元和单元中的内容是两个不同范畴的东西，因此，寻访单元和单元中的内容是两个不同的操作。